PaperDatabase: a text-based, verifiable knowledge base for assisted reading of scientific papers

Léonard Rivals Maksym Lytvynenko

October 3, 2026

Abstract

Preparing a PhD means reading, comparing and citing dozens of papers. A large language model (LLM) can help, but it works poorly on raw PDFs: it rereads the full text for every question, does not know where to look for a number, and mixes what a paper says with what it infers. PaperDatabase converts each PDF into structured text files (metadata, paginated sections, references, reading note, benchmark table) that an LLM agent, here Claude Code, walks from summary to detail. A passage-retrieval layer, in the RAG style, sits on top: a BM25 lexical index and a dense index (bge-small-en-v1.5) merged by Reciprocal Rank Fusion, with optional reranking. This layer generates nothing. It returns identified, paginated and replayable passages, plus a command that checks that a quotation, and in particular its numbers, does appear in the cited passage. Writing stays with the agent, bound by sourcing rules. Every answer of the agent opens with a sufficiency verdict, which confronts the requirements of the question with the passages read. A reproducible evaluation bench, with no LLM and no network, measures extraction, retrieval, verification, timings and, more recently, the written answers. On 101 papers (8 601 passages), hybrid retrieval with reranking reaches a recall@5 of 0.72 on a held-out question set that was never used for tuning: it was 0.80 on 45 papers, and the cause of this drop, corpus or code, is not yet established. In French, the best CPU mode retrieves 4 questions out of 8; an optional GPU engine (Qwen3) retrieved all of them on a smaller corpus. Verification rejects quotations with an altered number, with one documented exception. No retrieval score reliably detects that a question falls outside the corpus, but the agent, guided by the sufficiency verdict, recognises the 6 out-of-corpus questions of the answer evaluation without wrongly declaring any other one absent, and on the second run all its quotations are accepted. A real sourced-answer trial consumed about 2 500 tokens, whereas a single full text is about 12 000 tokens (median), and delegating three reading notes to subagents cost 41 % less than writing them directly (a single trial). The same code serves two other bases, on embedded vision on drones (87 papers), where retrieval meets its threshold, and on continual learning for predictive maintenance (56 papers).

Contents

1 Introduction **3**

2 Design principles **4**

3 Architecture and pipeline **4**
3.1 Acquisition and ingestion 4
3.2 Data model 5
3.3 Several bases 6
3.4 Modules 7

4 The retrieval layer (RAG) **7**
4.1 A RAG without generation 7
4.2 Passage chunking 7
4.3 Two indexes and a fusion 8
4.4 Commands 9
4.5 Answer protocol: the `/paper-ask` skill 10
4.6 An optional GPU engine and open-access acquisition 11

5 What the software brings **12**

6 Tests and evaluation **14**
6.1 Unit tests 14
6.2 The `pdb eval` bench 15
6.3 Extraction quality 16
6.4 Passage retrieval 17
6.5 Detecting that a question falls outside the corpus 19
6.6 Quotation checking 19
6.7 Timings 20
6.8 Real end-to-end trial 20
6.9 Evaluating the written answer 20
6.10 A second corpus: the drone base 22
6.11 Synthetic benchmark (`pdb synth`) 23
6.12 Threats to validity 23

7 Limitations and outlook **24**

8 Conclusion **25**

A Glossary of acronyms **27**

B Glossary **28**

C Commands **30**

1 Introduction

Context. This work stems from a concrete need: preparing a PhD proposal on embedded vision on FPGAs (YOLO detectors, autoencoders, continual learning). The field publishes many numbers (frames per second, GOPS, watts, LUT, DSP and BRAM resources), measured under heterogeneous conditions. Comparing them requires knowing where each value comes from. The corpus of this use case serves as a test bed, but the tool does not depend on the domain: the same code now serves two other bases, one on embedded vision on drones, the other on embedded continual learning on microcontrollers for predictive maintenance. What depends on the topic (axes, classification rules, tags, scope) is declared in each base's own files.

Problem. An LLM agent given PDFs has three flaws:

1. *cost*: it rereads full texts (about 12 000 tokens each, median) to answer a question that fits in three sentences;
2. *traceability*: a number or a quotation does not carry the exact place it comes from, and a paraphrased quotation passes for a verbatim one;
3. *confusion between read and inferred*: the agent fills gaps with its general knowledge without saying so.

Approach. PaperDatabase does not try to replace the agent. It gives it a base in which every piece of information carries its source, and deterministic commands that answer factual questions without calling an LLM. Generation is limited to questions that need a synthesis, and it works from outputs that are already sourced.

Contributions.

- An idempotent PDF → structured text pipeline (GROBID, PyMuPDF fallback, ligature repair, table reconstruction) that never guesses missing metadata (section 3).
- A fully textual data model, validated at every reindex, where every numerical value carries its source section and a confidence level (section 3.2).
- A hybrid retrieval layer without generation, organised in three cost levels and equipped with automatic quotation checking (section 4).
- A reproducible evaluation bench, with no LLM and no network, that measures the quality of extraction, retrieval and quotation checking, as well as timings, with a held-out question set never used to tune any parameter; and a unit-test suite that runs without models (section 6).
- An answer protocol that states the requirements of a question before searching and opens every answer with a sufficiency verdict (`suffisant`, `partiel`, `absent`), and an evaluation of the written answers themselves, scored without an LLM (sections 4.5 and 6.9).
- Atomic evidence (`claims.jsonl`), each item linked to a passage and checked at reindex, and explicit comparison conditions in the benchmark table (section 3.2).
- An optional GPU retrieval engine (Qwen3), built into the same command, and user-approved open-access paper acquisition (section 4.6).
- A separation of code and data that keeps several isolated bases, each with its own profile, rules and evaluation bench, and lets them exchange papers (section 3.3); the bench is reproduced on a second corpus (section 6.10).
- A log of the cost of the subagents to which the main agent delegates tasks, and a first paired measurement of what delegation saves (section 5).

2 Design principles

The final consumer of the base is not a search engine. It is an LLM agent that already has file-reading and text-search tools (`grep`, `read`). On a corpus of a few dozen to a few hundred papers, a well-named directory tree, an index and a filterable CSV are enough for most questions, and they remain readable, diffable from one version to the next and repairable by hand. Four rules follow.

1. **Everything is text.** No database and no binary format, apart from the original PDF, kept as evidence. The retrieval index (section 4) is a derived, rebuildable cache, outside the versioned tree.

2. **Every number is traceable.** Every value in the benchmark table carries the section, table or figure it comes from. A value not reported by the authors is written `n/r`, never an estimate.

3. **Read from summary to detail.** The agent reads the highest layer that answers and only goes down when needed (table 1). It never reopens the PDF.

4. **The schema is coded in a single place** (`lib/schema.py`) and checked at every reindex, which stops at the first discrepancy.

Table 1: Reading layers, from summary to detail.

Layer	File or command	Used to
1	`INDEX.md` (generated)	know which papers exist, by axis, with or without a note
2	`benchmarks.csv` via `pdb bench`	find a number, with its source section; `--comparable`: know which rows can be compared
3	`papers/<id>/fiche.md`	read the critical analysis of a paper
3 bis	`papers/<id>/claims.jsonl`	find a statement of the paper, already linked to its passage
4	`pdb search`, `show`, `verify`	find and quote the 2 or 3 passages that answer
–	`fulltext.md`	full text, as a last resort

An instruction file (`CLAUDE.md`) describes this reading order to the agent, together with working rules: never invent a reference, write `n/r`, separate what is read from what is inferred, flag preprints and non-comparable metrics.

3 Architecture and pipeline

3.1 Acquisition and ingestion

`bin/add.py` fetches an arXiv paper (metadata through the API, then the PDF), resolves a DOI to its arXiv version through OpenAlex (Priem et al., 2022), or copies a local PDF. Paywalled papers are dropped in by hand: the tool bypasses no access control.

`bin/ingest.py` sends each PDF to GROBID (GROBID contributors, 2008–2026), which returns a TEI structure (header, sections, figures, bibliography). Without GROBID, a PyMuPDF fallback splits the text heuristically on numbered headings and records this in the metadata. The pipeline then applies, in order:

- *metadata merging* by order of trust: publisher citation export, then arXiv, then GROBID. An unknown field stays `null`;

- *cleaning*: repeated headers and footers, page numbers, bibliography moved out of the body into `refs.jsonl`;

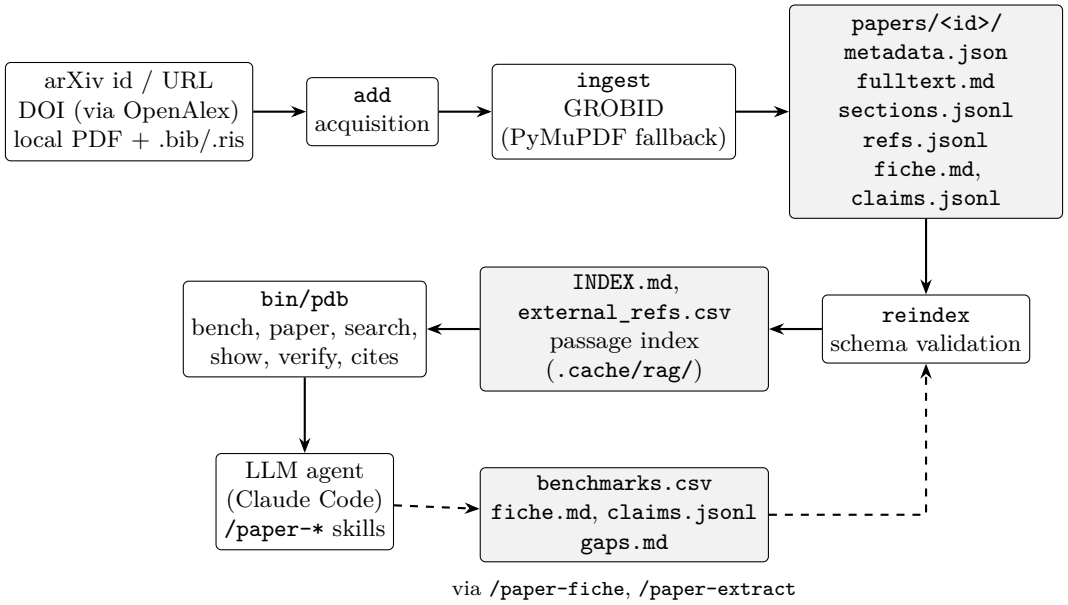


Figure 1: Data flow. Solid arrows are automatic; dashed arrows go through the agent, which writes reading notes, atomic evidence and benchmark rows from the extracted text.

- *repair of lost ligatures* (« classi cation » for « classification »): a word is rejoined only if the result is known to the corpus or to a built-in list and the fragment is not;
- *tables*: a table for which GROBID kept only the caption goes through two successive fallbacks. PyMuPDF first looks for it on its page (ruled tables). Docling (Auer et al., 2024) comes next, on request (`ingest --reparse --tables docling`), with optional optical character recognition (OCR) for tables published as images. A reconstructed body is labelled "reconstructed by PyMuPDF / Docling: to be checked", and a table left empty is reported by `pdb status`. Each table keeps its page, its frame on the page (`bbox`) and the extractor that produced it (`table_source`). Seven quality signals, computed at read time, flag suspect tables: empty cells, a varying number of cells from one row to another, no header, doubtful OCR characters, glued numbers, lost columns, and a caption matched by rank rather than by number (`pdb status --tables`);
- *misclassified paragraphs*: a paragraph that GROBID took for a figure is returned to the body when it looks like prose.

Ingestion is idempotent, and a re-extraction (`--force`, or `--reparse` from the stored TEI) never overwrites a reading note already written nor a classification corrected by hand.

3.2 Data model

Each paper becomes a folder `papers/<paper_id>/`. The identifier is stable and readable (`arxiv-<id>-<slug>` or `<year>-<author>-<slug>`). The extracted text exists in two forms: `fulltext.md`, human-readable, and `sections.jsonl`, one line per section with its type (abstract, section, figure, table) and its page. The latter is the raw material of the passage index.

The file `benchmarks.csv` has one row per reported configuration: a paper that presents three variants yields three rows. Besides the measurement columns (model, platform, quantisation, FPS, GOPS, latency, power, LUT, DSP, BRAM, dataset, toolflow), three columns are used to judge each value:

- `source_section`: section, table or figure of origin;

- `confidence`: `high` (read from a table), `medium` (read from the text) or `low` (inferred, to be rechecked);
- `power_scope`: scope of the reported power (`chip`, `board`, `board-dynamic`, `estimated`, `unspecified`). Two powers with different scopes cannot be compared: a board can draw several watts at idle, which a "board" measurement includes and a "chip" measurement excludes.

These twenty columns do not say everything about the conditions of a measurement. Seven *optional* columns, added on request (`pdb bench --add-columns`), complete them: the passage that contains the value (`chunk_id`), the exact definition of the metric, the evaluated split, the input resolution, the batch size, the measurement protocol and the value as written when it was converted. When absent, an optional column is `n/r`: an existing base stays valid without migration, and no current row fills them yet. `pdb bench --check` verifies that every number of a row that cites a passage does appear in that passage, and `pdb bench --comparable` groups the rows by task, dataset, split and metric (or power scope) before any comparison between papers. It flags the conditions that vary within a group, those that are missing, and the impossible comparisons. On the fpga base, the 56 detection rows on COCO, from 7 papers, form 9 groups, none of which gathers two papers: none of these values is directly comparable to another.

Reading notes (`fiche.md`) follow a fixed template (problem, method, results, limitations, relevance) with a mandatory "reading status" section that separates what was read from what was inferred. They are written by the agent (`/paper-fiche` command) and reviewed by the user.

A note summarises; its numbers carry their source, but its qualitative statements (method, limitation, hypothesis) do not. The `claims.jsonl` file of a paper holds its atomic evidence: one statement per line, with its type (result, method, limitation, hypothesis, context), its voice (the authors, or a deduction by the reader), the source passage, the exact excerpt that carries it, and optionally its conditions and the benchmark row that transcribes it. The `/paper-extract` skill writes them. At every reindex, each excerpt is checked by `verify` against its passage: an absent excerpt or one with wrong numbers is an error, and the reindex fails. A number of the statement that is absent from the source section is flagged. First trial, on the FINN paper: 32 items, 32 verified excerpts, a single note, a legitimate one (value taken from a neighbouring passage); an error introduced on purpose (altered number, wrong passage) is rejected. The other papers have not been processed yet, and retrieval does not read this evidence yet.

3.3 Several bases

The same code serves several knowledge bases, one per topic. The repository holds three (table 2): the *fpga* base of the use case, which serves as the reference and now covers a thesis on safe and certifiable embedded AI on FPGA; a *drone* base (embedded vision on drones: detection, accelerators, co-design of computing, energy and flight), created later on a neighbouring topic; and a *stage* base, on embedded continual learning on microcontrollers (MCU) for predictive maintenance, built from the bibliography of an internship. The latter is the furthest from the original domain: its founding papers (Mahalanobis, 1936, scanned and run through OCR) have no hardware measurement, and its benchmarks concern bearing and turbine datasets rather than accelerators. The repository root holds only the code. Each base is a `bases/<name>/` folder with its own papers, `benchmarks.csv`, gap map, passage index and caches; the commands, the Python environment and the agent's skills are links to the code.

What depends on the domain fits in two files per base. `base.json` declares the watch axes and regular-expression rules that put a paper in an axis, give it tags and decide whether it is on topic (the last test is used by discovery). `profile.md` describes the topic and scope (included, excluded) in prose; the `/base-profile` skill proposes refinements drawn from the reading notes, and writes nothing without approval. Each base also has its own `CLAUDE.md`, which carries the domain's working rules: for the drone base, stating the power range of a result, or telling ground-to-air

Table 2: The three bases on 03/10/2026 (output of `pdb base list` and `pdb status`).

Base	Papers	Notes done	Benchmark rows	Indexed passages
fpga (reference)	101	101	299	8 601
drone	87	86	352	5 361
stage	56	56	100	3 640

from air-to-air detection apart. The columns of `benchmarks.csv` remain common to all bases; the optional columns (section 3.2) are added base by base. The figures of `pdb plot` adapt as well: a `plot` block in `base.json` declares the tasks and platform families to distinguish and the figures to omit. The stage base, with no hardware measurements and no question sets, has 8 figures, versus 16 for fpga. `pdb plot --check` proposes this block from the content of the base, and `reindex` regenerates the figures that have become stale.

The active base follows from the context: the `$PDB_ROOT` variable (set by `pdb --base <name>`), otherwise `$PDB_BASE`, otherwise the first folder containing `base.json` walking up from the current directory, otherwise the default base of a local registry (`~/.config/pdb/bases.json`), otherwise the reference base. Two agent sessions opened in two bases therefore each work on their own.

Bases stay separate, but a paper can move from one to another. `pdb base scan` searches the other bases for papers useful to the current one: those it already cites without having them, those that match its rules, and references cited on both sides. `pdb base import` copies a paper without re-extracting it and sends its reading note back for review, since its relevance depends on the topic of the receiving base.

3.4 Modules

All commands go through a single entry point, `bin/pdb`. Their output follows the same conventions: TSV by default, a header line without banner, notes prefixed by `#`, short paper identifiers (`1612.07119`, `2022-feng`) with their legend at the bottom, and an exit code that distinguishes "results", "no result" and "argument error".

4 The retrieval layer (RAG)

4.1 A RAG without generation

In a *Retrieval-Augmented Generation* system (Lewis et al., 2020), the LLM receives, together with the question, the passages of a corpus that answer it, and writes from them. PaperDatabase separates the two halves. The `bin/pdb` commands do the retrieval, deterministically and without calling an LLM. The agent calls them, reads their output and writes only when a synthesis is needed. This choice adds neither an API key nor a cost: the generation is done by the agent already in use.

Three goals guide the design, in order of priority: *verifiability* (every returned item carries a replayable source), *token economy* (answering without reading full texts), and *access by meaning* (finding "continual learning" when searching for "incremental learning", which `grep` cannot do).

An N0 or N1 output is often already an answer: a filtered table or three sourced passages are enough for a factual question, and the agent can pass them on without rephrasing. Level N2 adds no information: it links what N0 and N1 returned, and labels its inferences as such.

4.2 Passage chunking

The starting unit is an entry of `sections.jsonl`, which already follows paragraphs. An entry longer than 250 words is split into 200-word windows with a 40-word overlap, because embedding models truncate beyond 256 to 512 tokens. Each passage gets a replayable identifier, `<section>.<k>`

Table 3: Main modules of `lib/`. Everything is written in Python; only dense retrieval, the GPU engine, the figures and the visualisation have optional dependencies.

Module	Role
`tei.py`, `doc.py`, `pdftext.py`	reading GROBID output, common representation, PyMuPDF fallback
`ligatures.py`, `tables.py`, `docling_tables.py`	ligature repair, table reconstruction (PyMuPDF, then Docling), quality signals
`citation.py`, `openalex.py`	`.bib`/`.ris` exports, OpenAlex client with cache
`schema.py`	data-model validators, the only place where it is coded
`refs.py`	reference matching (arXiv, DOI, normalised title), internal citations
`query.py`	structured queries without LLM (`bench`, `paper`, `cites`), `--check`, comparable groups
`rag.py`	passages (tables split by rows), BM25, dense index, fusion, reranking, quotation checking; optional Qwen engine
`claims.py`	checking of atomic evidence (`claims.jsonl`) against its passage
`rag_qwen.py`, `rag_models.py`	GPU engine: pinned Qwen3 models (shared with `pdb search`), FTS5 evidence index of `pdb rag`
`rag_external.py`	staged open-access acquisition (OpenAlex, arXiv), import after approval
`evaluation.py`	evaluation bench (`pdb eval`): extraction, retrieval, `verify`, timings, final answers
`agentlog.py`, `agentcost.py`	subagent log, cost calibrated on Claude Code, savings from delegation
`status.py`	state of the base and next actions
`base.py`, `registry.py`	active base, reading `base.json`, classification by its rules; registry of bases
`bases.py`, `xbase.py`, `vizbases.py`	creating and checking bases, scan and import across bases, Bases tab of the visualisation
`plotdata.py`, `plotfig.py`, `plotrun.py`	static figures (`pdb plot`): benchmarks, evaluation, corpus; regeneration of stale figures
`graph.py`, `lineage.py`, `vizfig.py`, `embed.py`	3D visualisation and citation lineage (secondary tool)

(for example `1612.07119#019.3`), and keeps its paper, section, page and the SHA-1 hash of its text. The indexed text is prefixed with the section title, which adds context without changing the returned text. A table longer than 250 words is split differently: by whole rows, each window repeating the abridged caption and the header row, so that a value is never separated from its unit or its column. Across the three bases, 181 tables are concerned; they yield 610 windows (median 186 words) with no loss of words. The fpga base now has 8 601 passages.

4.3 Two indexes and a fusion

Lexical index. A hand-written BM25 (Robertson and Zaragoza, 2009) ($k_1 = 1.5$, $b = 0.75$, lower-case alphanumeric tokens) finds exact terms: board names (`ZCU104`), model names (`YOLOv5n`), acronyms and numbers.

Dense index. The `BAAI/bge-small-en-v1.5` model (Xiao et al., 2024) (384 dimensions), loaded with sentence-transformers (Reimers and Gurevych, 2019), encodes each passage into a normalised vector. It finds paraphrases and, to some extent, queries in another language. The query gets the instruction prefix recommended for the model; passages do not.

Table 4: Three query levels, from cheapest to most expensive. One moves up a level only if the previous one is not enough. Costs measured on the current corpus.

Level	Source	LLM	Typical cost
N0 – structured query	`benchmarks.csv`, metadata	no	100–500 tokens
N1 – passage retrieval	passage index	no	400–800 tokens, +300 per opened passage
N2 – synthesis	N0 and N1 outputs only	yes	that of the writing

Fusion. Both rankings are merged by Reciprocal Rank Fusion (Cormack et al., 2009):

$$\text{score}(d) = \sum_{i \in \{\text{BM25, dense}\}} \frac{1}{60 + \text{rank}_i(d)}. \tag{1}$$

This method does not depend on score scales, needs no tuning and stays deterministic. Ties are broken by the smallest identifier: the same query on the same index always gives the same ranking.

Absolute relevance. The RRF score is a rank, not a relevance measure: `search` always returns k passages, even when the base does not cover the topic. The output shows the cosine between the query and each passage. A first version warned (`# pertinence faible`, "low relevance") when the best cosine fell below a threshold calibrated by hand on a few queries. The evaluation bench showed that this warning never fired, and that no threshold, neither on the cosine nor on a cross-encoder score, separated out-of-corpus questions from the others (section 6.5). It was therefore removed, because it gave false reassurance: the agent judges, from the snippets, whether the base answers, and says so otherwise instead of filling the gap.

Reranking (option). `--rerank` reorders the top 30 results with the `ms-marco-MiniLM-L-6-v2` cross-encoder, a MiniLM (Wang et al., 2020) trained on MS MARCO (Nguyen et al., 2016) following Nogueira and Cho (2019). It costs about 1.4 s per warm search. It is not enabled by default, for the reasons given in section 6.4.

Storage. Vectors are stored in `.cache/rag/<model>.npz`, keyed by `chunk_id:sha1`. Updates are incremental: only new or modified passages are encoded (about 8 min for a full build of the 8 601 passages of the fpga base on CPU, i.e. 18 passages per second; 0.2 s with no change). Passages and BM25 are recomputed by each command in under a second. Without sentence-transformers, or if the index is stale, `search` falls back to lexical only and says so. At this scale, an exhaustive numpy search takes a few milliseconds: a dedicated vector database would bring nothing before roughly 10^5 passages (estimate, not measured).

4.4 Commands

`pdb bench` filters `benchmarks.csv` (substrings, numerical bounds, non-empty columns, sorting, aggregation with `--count`). The source columns (`source_section`, `confidence`, `power_scope`) are always shown, even when not requested, and a low-confidence row is prefixed with `!`.

```
$ bin/pdb bench --platform kv260 --cols model,quantization,fps,power_w
paper       model                 quantization  fps  power_w  power_scope  source_section  confidence
2507.18174  YOLOv5 (variante n/r)  INT8          9    3.5      unspecified  Table I         high
# ids: 2507.18174=arxiv-2507.18174-real-time-object-detection-classification
```

`pdb search` returns one line per passage: identifier, section, page, cosine and a snippet of about 160 characters centred on the rarest query term (rare terms in guillemets). Real output, shortened snippets:

```
$ bin/pdb search "power measured at the wall versus the FPGA rails" -k 3
chunk              section                       page  cos   snippet
1612.07119#022.0   5. EVALUATION / 5.2.2. ...     8     0.78  ...both the «wall» «power» and «FPGA» «power
    ↪ » readings...
2025-hozhabr#051.1  V. RESULTS ANALYSIS ... / B. ...  31    0.77  ...computational «power», or throughput, «
    ↪ measured» in...
1612.07119#019.3   5. EVALUATION / 5.1. ...       7     0.73  ...PMBus interface to monitor the «FPGA» «
    ↪ power» supply «rails»...
```

`pdb show` prints the exact text of a passage with its header (section, page, start of the SHA-1) and, on request, its neighbours. `pdb verify` checks that a quotation appears in the cited passage. Both texts are normalised (ligatures, case, hyphenation, everything that is neither letter nor digit), then compared by exact search or, failing that, by the best `difflib` window. The verdict is one of five (the tool's labels are in French):

- `OK`: the quotation appears as is, after normalisation;
- `APPROX`: similarity of at least 0.85 (close paraphrase);
- `NOMBRES` ("numbers"): similarity of at least 0.85, but the numbers of the quotation are not those of the passage;
- `ABSENT`: similarity below 0.85;
- `MODIFIÉ` ("modified"): the passage has changed since its SHA-1 was recorded.

Only `OK` and `APPROX` give a zero exit code. The `NOMBRES` status comes from the evaluation bench. In a long sentence, a changed number keeps the similarity above the threshold, and the first version thus accepted every quotation with an altered number (section 6.6). The added check requires the numbers of the quotation to form a contiguous run of the passage's numbers. Each must appear with its neighbouring words: "Section 3 reviews" is rejected even if the passage contains a 3 elsewhere. Since normalisation removes punctuation, this check is also what tells "94.9 %" from "949 %".

A quotation absent from the cited passage but present elsewhere in the same paper is reported with the identifier where it was found, and the command exits with an error: the source was wrong.

```
$ bin/pdb verify 1612.07119#019.3 "P-wall using a wall-power meter for the whole board power consumption"
APPROX  0.95  1612.07119#019.3
```

Finally, `pdb paper` gives the identity of a paper (authors, venue, preprint or not, note status), and `pdb cites` / `cited-by` walk the citation graph internal to the corpus, rebuilt by matching references on arXiv identifier, DOI or normalised title.

4.5 Answer protocol: the `/paper-ask` skill

The `/paper-ask` skill gives the agent a fixed procedure and only allows it to use the `bin/pdb` commands and file reading:

1. before any search, state 2 to 5 requirements, what a complete answer must establish (a value, a platform, a condition, a comparison), without going further than the question asks;
2. numerical question: `pdb bench` first (N0), and `bench --comparable` before comparing two papers;
3. otherwise, or in addition: `pdb search --rerank` with one or two English phrasings; if no snippet answers, say that the base probably does not cover the topic instead of filling the gap;
4. `pdb show` on 2 or 3 passages at most, never on the full text;

5. answer with `[chunk_id]` citations, and `pdb verify` on every verbatim quotation: an `ABSENT` quotation is removed, a `NOMBRES` quotation is corrected from the exact text or removed, a wrong identifier is corrected, and an `APPROX` quotation is kept only if presented as a paraphrase;

6. mark each requirement `couvert` ("covered", verified passage), `partiel` ("partial", neighbouring passage: other platform, other batch, other power scope) or `absent`, and derive the verdict from them: `suffisant` ("sufficient") if all are covered, `absent` if none is, `partiel` otherwise;

7. imposed structure: the verdict and the requirements first, then **Read** (sourced), **Inferred**, **Not found in the base**, flagging low-confidence values, preprints and heterogeneous power scopes.

The verdict moves the question "does the base cover this topic?" away from a retrieval score, which does not settle it (section 6.5), toward the content of the passages read. An `absent` verdict concerns the base, not the literature: the answer says so and suggests `/paper-discover`. A `--record` mode writes the answer as JSON, in the form of cited statements, for the evaluation of answers (section 6.9).

4.6 An optional GPU engine and open-access acquisition

Engine. A second retrieval chain was developed in parallel, for a machine with a GPU (measurements made on a 32 GB RTX 5090): `Qwen3-Embedding-4B` for the dense index and `Qwen3-Reranker-4B` for reranking, in BF16 with pinned model revisions. It is now an *optional engine of* ***`pdb search`***: it encodes exactly the same passages, with the same identifiers, so that `show`, `verify`, `/paper-ask` and the evaluation of answers do not change. The CPU engine remains the default. `pdb gpu on` enables the GPU engine for the machine (a setting outside the bases), downloads the models and reindexes; `--gpu` or `--cpu` forces one for a single search. Without CUDA, without the required libraries or without Qwen vectors for the base, `search` stays on the CPU and says so: a question never fails for lack of a GPU. `pdb eval --rag --gpu` writes a report with the same keys as the CPU engine, which makes it possible to compare the two engines with `--compare`.

The `bin/pdb rag` command keeps what remains specific to the original chain: an SQLite index (FTS5 and Qwen vectors) of *typed* evidence, which holds, next to the abstracts, sections, tables and figures of the original text, the finished reading notes and the rows of `benchmarks.csv`, each labelled as transcription or note. A note or a benchmark row is for navigation, and the agent must go back to the original passage (`rag evidence ID --parent`) before quoting. `rag benchmarks` filters numbers with no cap on results; `rag check-answer` checks, for an answer recorded in JSON, that each quoted excerpt appears in the cited evidence, without yet checking numbers as `verify` does. The `/paper-research` skill frames this route.

Acquisition. When the corpus is not enough, `rag acquire` looks for open-access full texts (OpenAlex, arXiv), downloads them into a private staging directory, extracts them and deduplicates them by DOI, arXiv identifier and document hash. Nothing enters the base at this stage: the agent presents each retained paper (title, year, excerpt) and calls `rag approve` only for those the user accepts. An imported paper gets a `todo` note and never overwrites a paper already present. A budget bounds each session: 8 searches, 8 PDFs, 50 MB per PDF, 200 MB and 15 minutes in total. An unavailable PDF is treated as a limitation of the search, not as evidence of absence.

Measurement. The two engines were compared on the question sets of section 6.4, with the same expectations and the same metrics (table 5). This comparison dates from 26/09/2026, on the 28-paper corpus, with the original `pdb rag` chain: it was redone neither on the current corpus

nor with the engine built into `pdb search`, as the machine used for the later measurements has no GPU.

Table 5: CPU engine (bge-small) versus GPU engine (Qwen3), same question sets, 28-paper corpus. Bold: best value per set.

Set	Engine	recall@5	recall@10	MRR	nDCG@10	s / question
dev (20)	bge-small hybrid + reranking	**1.00**	**1.00**	**0.85**	**0.46**	∼1.4
	Qwen hybrid	0.90	0.90	0.78	0.37	0.7
	Qwen hybrid + reranking	0.80	0.80	0.76	0.35	3.7
holdout (25)	bge-small hybrid + reranking	0.80	0.80	**0.78**	**0.41**	∼1.4
	Qwen hybrid + reranking	**0.88**	**0.92**	0.75	0.35	3.3
FR (8)	bge-small dense (best mode)	0.62	0.75	0.46	0.19	0.1
	Qwen dense	**1.00**	**1.00**	0.63	0.22	0.7
	Qwen hybrid + reranking	**1.00**	**1.00**	**0.88**	**0.31**	3.8

The only clear gap is French: Qwen retrieves all 8 questions without rephrasing them in English. On the held-out set, it gains 2 questions out of 25 at recall@5, with a comparable MRR. On the dev set, it does worse, which is partly expected since this set was used to tune the first engine. Its lexical channel (FTS5) is weaker than the first engine's BM25 (dev 0.80 versus 0.95; French 0.00). The Qwen reranker is also the best out-of-corpus signal measured (section 6.5). It comes at a cost: a GPU (each model takes about 8 GB), and models reloaded for each query, i.e. 3 to 4 s per question with reranking. The CPU engine therefore remains the default; the GPU engine is an aid for French questions and vocabulary mismatch.

5 What the software brings

Token savings. Table 6 compares, for four common tasks, reading the files directly and going through the commands. Orders of magnitude are measured on the current corpus (file size divided by four).

Table 6: Token cost of common tasks (orders of magnitude).

Task	Without the retrieval layer	With
A number on a platform	read the whole `benchmarks.csv`: ∼27 000	filtered `bench`: 100–400
Find where a topic is discussed	`grep` over sections, which misses synonyms	`search -k 8`: ∼600
Read the evidence for a claim	open `fulltext.md`: ∼12 000 (median)	`show` on 1 to 3 passages: 300–900
Check a quotation	reread the passage	`verify`: ∼20

Verifiability. Every returned item can be replayed: a passage by its identifier (`show`) and its page in the PDF, a quotation by `verify`, a number by its `bench` row and source section, a ranking by the determinism of the fusion. An empty result is presented as a statement about the base, not about the literature. Paraphrases, cross-paper connections and inferences cannot be checked automatically: the agent must label them as inferred.

Beyond retrieval. The same base serves other PhD-preparation tasks:

- extracted references feed `external_refs.csv`, which ranks works missing from the base by the number of corpus papers that cite them: these are the next readings;

- `pdb discover` (`/paper-discover`) queries OpenAlex (incoming citations, search by axis or by gap) and proposes candidates, added only with the user's approval;
- `/paper-gaps` keeps a map of uncovered areas (`gaps.md`), each entry supported by paper identifiers;
- state-of-the-art reviews in LaTeX were written by the agent from the notes of the fpga and drone bases (sections by axis, glossaries, links to the papers), then proofread; every statement in them points to a paper of the base;
- a local visualisation app (3D corpus graph, citation timeline) gives the user an overview; the agent does not use it;
- `pdb plot` produces static figures (PNG, PDF or SVG) from `benchmarks.csv`, the latest `pdb eval` report and the corpus metadata: throughput versus power, GOPS/W by precision, FPGA resources, accuracy versus throughput, retrieval quality, years, axes, citations, gaps. Each figure states its source and sample size in a footer (for example 82 rows out of 299, `n/r` values being excluded), and `figures/manifest.json` lists them. Figure 2 is an example; heterogeneous power scopes are distinguished there rather than mixed.

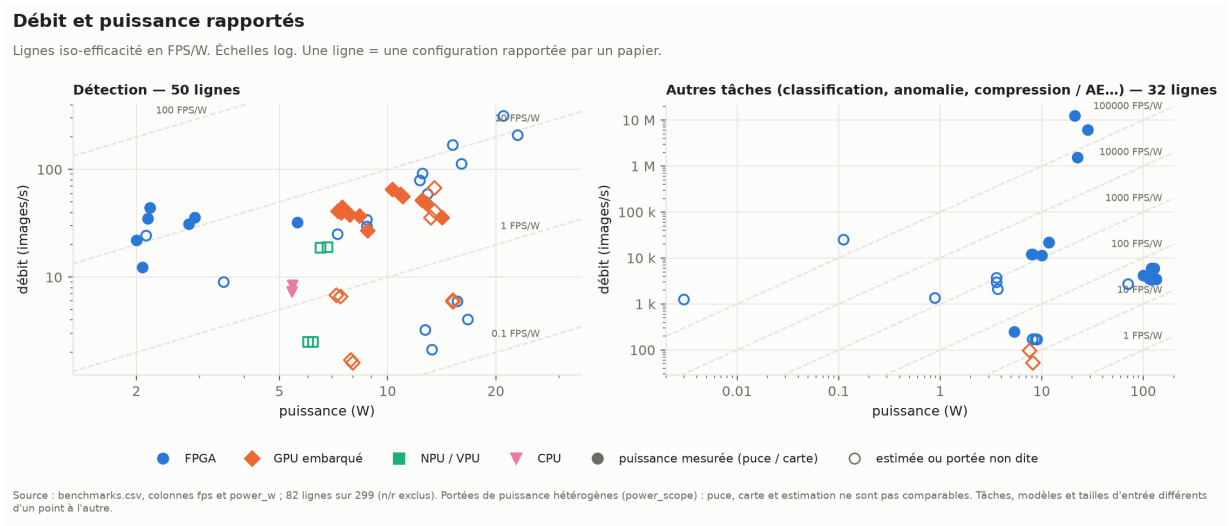


Figure 2: Output of `pdb plot bench` on the fpga base: reported throughput and power, one configuration per point (82 rows of `benchmarks.csv` out of 299), detection on the left, other tasks on the right. Filled markers: measured power; hollow: estimated or unspecified scope. Points cannot be compared with one another without reading the scope of the measurement (section 3.2, `bench --comparable`).

Delegating to subagents. The main agent can hand a task to a subagent that starts without its context: one note per agent, one paper to extract, a DOI lookup. The repository rules set the model by task: the lightest for mechanical work (adding, reindexing, checking metadata), an intermediate one for reading and extraction, the most capable for synthesis and judgement. Two agents never write the same file, and `benchmarks.csv`, `INDEX.md` and the reindex stay with the main agent, which verifies what is reported to it. To know whether this pays, a Claude Code `SubagentStop` hook logs every subagent (tokens, cost, duration, tools) in `.cache/agents.jsonl`, and `pdb agents --report` summarises it. The cost derived from the transcripts underestimated output tokens; it is calibrated on the cost that Claude Code counts for the session. Over 72 logged subagents (126 M tokens, $81), delegation would have saved about 59 % of the cost of the same

work done directly. This figure is an *estimate*: the same turns at the main agent's rate, plus rereading the work at each following turn.

To calibrate it, the same task (three reading notes on YOLO detectors of the drone base) was done twice in fresh sessions, directly and then delegated to three subagents in parallel (table 7). The model predicts $2.21 for the direct session, which cost $2.30 (a 4 % gap). Delegation cost 41 % less and took 59 % less time. The trial is single, and the two sessions did not produce the same work: directly, the notes are longer and include 9 more benchmark rows. Part of the gap therefore comes from the work, not only from the rate, and quality was not scored. The experiment also revealed a defect of the log: the hook ran before the last turn of the subagent was written, which was missing from the count; it now rereads the transcript until it is complete.

Table 7: Paired experiment: three reading notes, direct and delegated (03/10/2026, a single trial).

	Direct	Delegated
Cost counted by Claude Code	$2.30	$1.35 (0.46 + 0.89 for the subagents)
Duration	324 s	133 s
Main agent turns	29	6 (+ 24 for the 3 subagents)
Main context at end of session	139k tokens	54k tokens
`benchmarks.csv` rows / size of the notes	24 / 23.5k chars	15 / 17.8k chars

State of the corpus. As of 3 October 2026, the fpga base has 101 papers, all with a finished reading note, 299 benchmark rows, 8 601 indexed passages and 37 recorded gaps, 29 of them supported by corpus papers (output of `pdb status`). The corpus has more than doubled since the previous version of this text (45 papers, 3 479 passages), largely with papers on the safety and certification of embedded AI; the notes and benchmark rows of the new papers were written by subagents, then verified by the main agent.

6 Tests and evaluation

6.1 Unit tests

The suite (`bin/pdb test`) has 286 tests in 26 files. It runs in under 30 seconds, without GROBID and without embedding models, thanks to deterministic stand-ins (table 8):

- a real GROBID (0.8.1) TEI output, kept as a fixture, for parsing;
- a fake encoder and cross-encoder, deterministic functions of the text, to test fusion, reranking and the incremental index without sentence-transformers;
- simulated OpenAlex responses for discovery and addition by DOI;
- a synthetic PDF produced by PyMuPDF (a caption followed by a ruled table) for table reconstruction, and hand-built grids for matching Docling tables;
- small synthetic corpora (including a dated citation chain) for the graph and the lineage;
- temporary bases and registry for resolving the active base, creating a base, and scanning and importing across bases;
- hand-built subagent transcripts, for the log and for the computation of the cost of delegation.

Visualisation and figure tests are skipped if their optional dependencies are not installed. The latest run (03/10/2026) passes all 286 tests, including the open-access acquisition test, which failed in the previous run because it could not load the Qwen tokenizer without network.

Table 8: Unit-test coverage (number of test functions per file).

File	Tests	What is checked
`test_plot.py`	29	platform families, precision classes, `n/r` exclusion, per-base `plot` block, stale figures, rendering on the real base
`test_query.py`	23	`bench` filters, sorting and aggregation, forced source columns, `--check`, comparable groups, short ids, `paper`
`test_tables.py`	19	table reconstruction (synthetic PDF), Docling matching, quality signals, caption numbers
`test_schema.py`	19	data-model validators, optional columns, heading hierarchy
`test_rag.py`	18	chunking (tables by rows), BM25, fusion, reranking, incremental index, simulated Qwen engine, `verify` and number check
`test_evaluation.py`	18	bench metrics on small built cases, scoring of final answers, per-base bench locations
`test_tei.py`	12	parsing of a real GROBID output
`test_openalex.py`	12	discovery, addition by DOI and by list (simulated responses)
`test_base.py`	12	active base, minimal `base.json`, invalid rules, classification, `plot` block
`test_refs.py`	11	reference matching, `external_refs.csv`, internal citations
`test_citation.py`	11	`.bib`/`.ris` exports: reading, matching, priority
`test_xbase.py`	10	scan across bases, import (header of the target base), base creation, Bases tab
`test_agentlog.py`	10	subagent log: deduplication, incomplete answer, hook that ran too early and is reread, outcome recorded (`--mark`)
`test_agentcost.py`	10	cost per model, calibration on Claude Code, verification, duplicated reads, compaction, estimated savings
`test_lineage.py`	9	lineage on a dated citation chain
`test_graph.py`	8	corpus graph on a mini-corpus
`test_bench.py`	8	synthetic benchmark: isolated corpus, evidence location, structured `claude -p` output, generator model check
`test_ligatures.py`	7	repair of lost ligatures
`test_viz.py`	6	figures and detail panel, without a server
`test_status.py`	6	state of the base, flagged tables, next actions
`test_ingest.py`	6	axes, tags and warnings kept on re-extraction
`test_claims.py`	6	atomic evidence: schema, duplicates, absent excerpt, altered number, wrong passage
`test_rag_qwen.py`	5	GPU engine in lexical mode: sources, numerical conditions, stale snapshot, isolated acquisition
`test_reindex.py`	4	benchmark notes, clean measurements
`test_embed.py`	4	embedding cache, wiring into the graph
`test_registry.py`	3	registry of bases, resolution order of the active base

Two safeguards complement these tests in use: `reindex` validates every file against the schema and stops at the first discrepancy, and a list of manual checks of the ingestion chain (metadata, section order, no leftover headers, fallback without GROBID, cross-checking a note's numbers against the PDF) is kept in the technical documentation.

6.2 The `pdb eval` bench

Unit tests check that the code does what it claims. They do not say whether extraction is good or whether retrieval finds the right passages. `bin/pdb eval` measures this on the real corpus, with no LLM and no network, and without writing anything to the base. It covers four parts: extraction, retrieval, quotation checking and timings; a fifth, the written answers, is described in section 6.9. A full run takes a few minutes on CPU. `--save` stores the report as JSON (dated, with the measured commit), and `--compare` reports the differences with the latest report, with an error code if a threshold of `eval/thresholds.json` is no longer met. The procedure is: save before a change (extraction, chunking, model, fusion), make it, then compare. The bench runs on the active

base: a base can have its own question sets and thresholds, and its reports are stored in its folder (section 6.10). The figures below are for the fpga base and come from a run of 03/10/2026 on 101 papers and 8 601 passages (`bases/fpga/eval/results/2026-10-03T2340-3925690.json`). They are compared, in parentheses, with the reference run of 28/09 on 45 papers and 3 479 passages (`2026-09-28T1039-76dc85e`): the question sets did not change, but the corpus more than doubled and the code evolved (table chunking, restructured search engine). A difference between the two runs therefore mixes these two causes.

6.3 Extraction quality

The bench first computes intrinsic metrics, which need no annotation. It then reuses two ground truths already present in the base: the hand-checked metadata and the values recorded in `benchmarks.csv` (table 9).

Table 9: Extraction quality (101 papers, all extracted by GROBID; in parentheses, the 45-paper run).

Metric	Value
Text coverage: share of PDF words (bibliography excluded) found in `fulltext.md`	mean 0.90; min. 0.57 (0.91; 0.81)
Tables whose body was extracted (584 tables, 110 reconstructed by Docling and 11 by PyMuPDF)	0.78 (0.88)
Remaining ligatures, per 10 000 words	0.00
Papers with undetected numbered sections	31 / 101 (13 / 45)
GROBID header alone versus `metadata.json`: title / year / DOI / arXiv	0.87 / 0.47 / 0.65 / 0.82
GROBID header alone: F1 on author names	0.84 (0.86)
`benchmarks.csv` values found in the cited table (1 044 values)	0.70 (0.65)
same, rows citing tables only (682 values)	0.74 (0.71)
`benchmarks.csv` values found anywhere in the text	0.79 (0.76)

- *Coverage* cannot reach 1: headers, affiliations, text inside vector figures and hyphenated words are deliberately excluded. It is read relatively, from one run or one paper to another. The minimum (0.57) is a 103-page thesis; the next two are regulatory documents of the European Union Aviation Safety Agency (EASA), which came in with the safety and certification axis.
- *Year and DOI* are low mostly by construction: the arXiv PDF often carries neither the date nor the DOI of the published version. That is why ingestion fills them from arXiv and from citation exports. The bench also revealed a real, still open defect: a publisher template DOI (MDPI) is not rejected.
- *Tables.* The share of tables with a body drops (0.88 → 0.78), and the drop comes entirely from the 56 new papers: the 45 old ones keep 25 tables without a body out of 233, the new ones have 103 out of 351, 29 of them in the two EASA documents. Docling had raised this share from 0.66 to 0.90 on 28 papers, with OCR for tables published as images; but the quality of OCR'd tables is uneven (a visual check of three tables found one clean, one incomplete and one partly unreadable), hence the "to be checked" label. The quality signals (section 3) flag 255 tables out of 584: 163 with an irregular number of cells, 107 with glued numbers, 82 with empty cells, 14 with doubtful OCR characters. The sampled tables of these last two categories show real defects (three values in one cell, lost first letters). The "with a body" rate measures the presence of a body, not the accuracy of the cells: a per-table ground truth remains to be built.

- *Recovered values go up* (0.65 → 0.70 in the cited table) after a drop when going from 28 to 45 papers. The benchmark rows of the new papers are therefore, on average, better anchored in their table. The bench does not say whether a missing value comes from extraction or from the agent's transcription; this is what `bench --check` will check row by row, once the rows carry their passage (`chunk_id`): none does yet.

6.4 Passage retrieval

Question sets. Each question is linked to the relevant sections of the corpus (identified by `paper#order`, independently of the window chunking). These sections are chosen by text pattern in the target papers (for example *reuse factor* in the two hls4ml papers). Questions rephrase the topic without repeating the pattern when possible: "removes duplicate bounding boxes" for non-maximum suppression. There are three sets:

- *dev*: 20 English questions, used for the design choices (model, fusion, reranking);
- *holdout*: 25 English questions, written afterwards and never used to tune anything. They covered the 12 papers absent from the dev set and include 2 questions on tables and 3 questions spanning several papers. Three expectations were widened after reading the returned passages, because those passages did answer the question (before that: recall@5 of 0.68 with reranking);
- *FR*: 8 dev questions translated into French by hand.

These sets were written for a corpus of 28 to 45 papers. Of the current 101, about 60 % of the papers are targeted by no question.

Metrics. recall@5 and recall@10 (at least one expected section among the top 5 or 10 passages), MRR (inverse of the rank of the first correct section), nDCG@10 with binary gain (Järvelin and Kekäläinen, 2002), which rewards finding *several* relevant sections, and paper@5 (at least one passage from a target paper among the top 5).

Table 10: Passage retrieval over 8 601 passages (101 papers, bge-small-en-v1.5 model), on 03/10/2026; in parentheses, the recall@5 of the 45-paper run. Bold: best value per set.

Set	Mode	recall@5	recall@10	MRR	nDCG@10	paper@5
dev (20)	lexical (BM25)	0.85 (0.95)	0.90	0.66	0.32	0.90
	dense	**0.90** (0.90)	0.90	0.69	0.34	0.90
	hybrid (default)	0.80 (0.90)	0.90	0.67	0.34	0.90
	hybrid + reranking	**0.90** (1.00)	**0.95**	**0.76**	**0.38**	**0.95**
holdout (25)	lexical (BM25)	0.56 (0.60)	0.64	0.50	0.23	0.60
	dense	0.64 (0.64)	0.64	0.49	0.29	0.76
	hybrid (default)	0.68 (0.72)	0.68	0.56	0.26	0.72
	hybrid + reranking	**0.72** (0.80)	**0.72**	**0.62**	**0.31**	**0.80**
FR (8)	lexical (BM25)	0.00 (0.12)	0.38	0.05	0.03	0.25
	dense	**0.50** (0.62)	0.50	0.26	0.12	**0.62**
	hybrid (default)	**0.50** (0.50)	**0.62**	**0.29**	**0.13**	**0.62**
	hybrid + reranking	0.38 (0.50)	0.50	**0.29**	0.10	**0.62**

Reading the results.

- *The holdout falls below the threshold.* With 101 papers, no mode reaches a recall@5 of 0.8 on the holdout any more: the best, hybrid with reranking, drops from 0.80 to 0.72, i.e. two

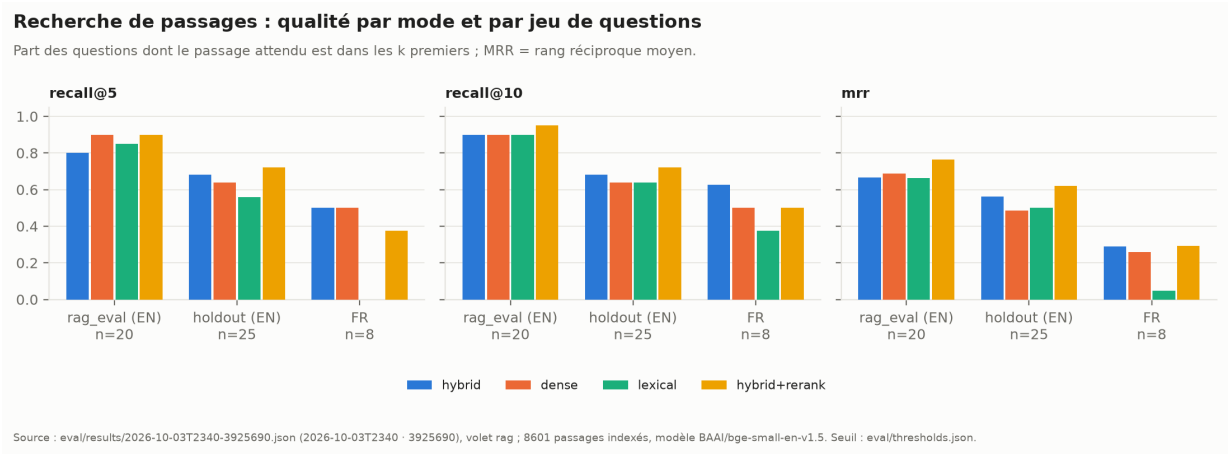


Figure 3: The numbers of table 10, plotted by `pdb plot eval` from the bench's JSON report. The dashed line is the acceptance threshold (recall@5 $\geq$ 0.8).

questions fewer. The dev set also loses (1.00 $\rightarrow$ 0.90), as does French (0.50 $\rightarrow$ 0.38). This is the most important result of this run, and its cause is not established: the corpus more than doubled, which adds passages close to the expected ones; tables are now indexed by rows; and the search code was restructured. The only change measured in isolation, table chunking, changes neither the dev nor the holdout. The planned diagnosis replays the sets on the index restricted to the 45 papers of the reference run, then the old code on the current corpus, to separate the effect of the corpus from that of the code. The threshold is not lowered, and nothing is tuned on the holdout.

- *The dev set was optimistic.* On the holdout, recall@5 loses 12 to 29 points depending on the mode, and lexical is the weakest. This confirms the expected bias: on the dev set, the expected sections had been chosen by keywords, which favoured BM25.
- *Reranking remains the best mode in English*, on both sets. It no longer compensates, on its own, for the competition of the new papers.
- *Real holdout failures* (with reranking): radiation tolerance (two questions), quantisation-aware training, memory of past examples, dataset distillation, and the two table questions (autoencoder resources in an hls4ml paper, FINN prototypes). For these two, the initial finding ("the text of a table is reduced to its caption") no longer holds: since Docling, these tables have a body and are short enough not to be split. The failure comes from vocabulary: the question says "resource utilization", "throughput", "autoencoder"; the table says "LUT", "Thr.put", "AE". Prefixing table passages with the paper title was tried, and does not help.
- *In French, lexical collapses* (recall@5 of 0.00), and the dense index holds up better, although the model is not designed as multilingual. Reranking, trained on English MS MARCO, degrades the ranking. The GPU engine (section 4.6) did much better on this set.
- *Decisions.* Hybrid remains the default mode of `pdb search`, because reranking degrades French, following a rule set before the measurement. `/paper-ask` enables it, because it always phrases its queries in English and the rank of the first correct answer matters when only 2 or 3 passages are opened.

In an earlier measurement (25/09/2026, dev set), the `all-MiniLM-L6-v2` model reached a recall@5 of 0.75 in dense mode versus 0.90 for bge-small, which motivated choosing the latter. The acceptance threshold is a recall@5 of at least 0.8 in English.

6.5 Detecting that a question falls outside the corpus

Can one tell, without an LLM, that the base does not cover a question? Two sets of 10 out-of-corpus questions (protein folding, blockchain, etc.) were written: one for calibration, the other for validation. The second was written before any cross-encoder measurement. Two signals are measured: the best cosine of dense retrieval, and the best cross-encoder score over the top 3 passages. For each, the threshold is chosen on the calibration data (dev set and first out-of-corpus set, from which a question half-covered by the corpus was removed) by maximising balanced accuracy, then applied as is to the validation data (holdout and second out-of-corpus set). The adoption rule was set before the measurement: on validation, at least 80 % of out-of-corpus questions flagged, and at most 5 % of corpus questions.

Table 11: "Out-of-corpus" signal: share of questions flagged on validation (10 out-of-corpus questions, 25 holdout questions), fpga base, 101 papers.

Signal	Threshold	Out-of-corpus flagged	Corpus wrongly flagged	Adopted
cosine, initial hand-set threshold	0.60	0.00	0.00	–
cosine, calibrated threshold	0.78	0.80	0.20	no
cross-encoder, calibrated threshold	2.12	0.90	0.32	no
Qwen3-Reranker-4B, 28 papers (section 4.6)	0.96	1.00	0.16	no

No signal meets the rule. With bge-small, an off-topic question gets a cosine as high as a relevant one: the initial calibration, made on a few queries, did not hold, and the warning never fired. The cross-encoder separates the medians better, but some corpus questions score very low, often those where retrieval already fails. The warning was therefore removed from `pdb search`. The GPU engine's reranker is the signal closest to the rule: it flags all 10 out-of-corpus questions, but also 4 of the 25 holdout questions (16 %, for 5 % allowed). It is not adopted either. The answer came from elsewhere: when it is the agent that judges, from the content of the passages read, it recognises the out-of-corpus questions (section 6.9).

6.6 Quotation checking

The bench draws 40 random passages (fixed seed) and builds four kinds of quotations: an exact sentence, which must be accepted; the same sentence with one word replaced, then with one number changed, and a sentence from another paper, which must be rejected. "Accepted" means a zero exit code (`OK` or `APPROX` in the cited passage).

Table 12: Acceptance rate of `pdb verify`, before and after the number check. "Before": 28-paper run; "After": 03/10 run on 101 papers, whose random draw differs (33 sentences with a changeable number instead of 38).

Quotation	Before	After	Statuses after
exact (40)	1.00	1.00	40 `OK`
number changed (33)	**1.00**	**0.00**	33 `NOMBRES`
word replaced (40)	0.97	0.97	39 `APPROX`, 1 `NOMBRES`
from another paper (40)	0.00	0.00	40 `ABSENT`

Before the check, `verify` accepted every quotation with a changed number, whereas the agent's working rules rely on it to guarantee quoted numbers. A first version of the check still let 6 through (a "4" recognised in "6.4", a "2" found elsewhere in the passage). Requiring neighbouring words fixes both cases, but not all of them. On the drone base (section 6.10), the 03/10 run accepts, for the first time, a quotation with a changed number: "before step 2" instead of "before

step 4", in a passage that contains "in step 2 and 3" elsewhere. The changed number and its neighbouring word do exist in the passage, but not at that place. This case is not fixed; it shows that the check guarantees the presence of each number with its immediate context, not its position in the sentence. Also still unhandled: a *word* replaced far from any number still yields `APPROX`. That is why `/paper-ask` keeps an `APPROX` quotation only when presented as a paraphrase.

6.7 Timings

Table 13: Timings measured on the fpga base (8 601 passages; medians of 3 repetitions, CPU without GPU).

Measurement	Value
Extraction outside GROBID, per paper (TEI → text, ligatures, tables)	0.01 to 0.13 s
Index loading (8 601 passages, 17 MB)	0.52 s
bge-small encoding	18 passages/s; full index ≈ 491 s
Warm `search`, p50 / p95: lexical	25 / 31 ms
dense	117 / 146 ms
hybrid	115 / 168 ms
hybrid + reranking	1 430 / 1 782 ms
Warm `verify`	2 ms
Cold commands, Python start-up included: `status`	0.61 s
`bench`, `paper`	0.09 and 0.08 s
`show`	0.75 s
`search`	6.6 s

The cost of a search comes almost entirely from start-up (PyTorch import, model loading): 6.6 s cold versus 0.1 s warm. This is the real cost of a call by the agent, which launches a command each time. With 2.5 times more passages than in the 28/09 run, warm search timings have barely moved, and index loading goes from 0.29 to 0.52 s. The `verify` time, which had gone from 4 to 246 ms without explanation, is back to 2 ms.

6.8 Real end-to-end trial

Question asked to `/paper-ask`, on the 28-paper corpus: "how do the papers measure power, and is it comparable?". The agent chained one `bench` query (57 rows, 8 papers), one search of 10 passages, reading 3 of them, 7 quotation checks and 8 identity queries. The whole amounts to about 9 kB read, i.e. about 2 500 tokens, without opening any full text. All 7 verbatim quotations came back `OK` at the first attempt. Retrieval found in one query the four measurement protocols of the corpus (PMBus and wall socket, multimeter at the board input, `tegrastats`, idle power measured before the tests). The Read / Inferred / Not found structure brought out two points to check in the base itself, including a paper that describes a careful measurement while no power row had been entered. The trial also revealed two defects, fixed since and each covered by a test: a short identifier truncated for a compound author name, and an aggregation that required an external script (hence `bench --count`). This trial was a single one, judged by the agent that ran it; the next section replaces it with a repeatable evaluation.

6.9 Evaluating the written answer

The previous parts measure the ranking of passages. They do not say whether the final answer recognises an out-of-corpus question, whether its quotations hold, or whether it finds the expected sources and values. `pdb eval --answers` measures this, still without an LLM: the answers are produced separately, then scored.

Protocol. Each question of an answer set carries a category, the requirements of a complete answer, the expected sources (sections or papers), the expected values with their unit, and the expected verdict. There are four categories: a sourced fact in a paper (*simple*), values with their conditions (*numerical*), a synthesis across several papers (*synthesis*), and a question close to the theme that the corpus does not cover (*absent*: polyps in endoscopy, sign language, sonar, EEG…, checked absent by full-text search). The `/eval-answers` skill hands each question to a subagent that receives only its text, no category, no sources, no expected verdict, and that answers with `/paper-ask --record`. The recorded answer is a list of statements marked read or inferred, each with its citations and their excerpts. The bench computes the accuracy of the verdict, the share of accepted citations (`verify` returns `OK` or `APPROX` in the cited passage, or the cited section is among the sources of the paper's `bench` rows), the share of "read" statements without any accepted citation, and the recall of the expected sources and values. A grid of human ratings (requirements, support, conditions, limitations, from 0 to 2) is planned: the automatic checks verify that the excerpts exist, not that they support the sentence.

First runs. The starting sets of the fpga base have 12 development and 12 held-out questions, 3 per category, written by the author of the code. Two runs were made on 02/10/2026, one answer per question, by Sonnet subagents; the second with the version of `/paper-ask` that states the requirements and the sufficiency verdict (section 4.5). Both runs were made in a degraded configuration: the dense retrieval and reranking models could not be downloaded from the container, so retrieval was lexical only (table 14).

Table 14: Written answers, fpga base (12 + 12 questions, lexical retrieval only). In parentheses: first run, before the sufficiency verdict.

Set	Correct verdict	Accepted citations	Unsupported statements	Expected sources	Expected values
dev (12)	0.75 (0.83)	1.00 (0.96)	0.00 (0.02)	0.87 (0.91)	1.00 (1.00)
holdout (12)	0.83 (0.92)	1.00 (1.00)	0.00 (0.00)	0.50 (0.67)	1.00 (1.00)

- *Out of corpus.* In both runs, the 6 *absent* questions are recognised as such, and no corpus question is wrongly declared absent. The answers say that the base does not cover the topic, without filling the gap with general knowledge. This is what no retrieval score could achieve (section 6.5). With 6 questions, it is a signal, not a measurement.
- *Citations.* In the second run, all citations are accepted and every "read" statement carries at least one.
- *Synthesis.* This is the weak category: 3 correct verdicts out of 6 in the first run, 2 in the second. The agent answers `partiel` where the set expects `suffisant`, because a requirement of the question (memory budget of each method, comparability of the measurements) is only partly covered by the corpus. These answers are defensible; the boundary between sufficient and partial is to be clarified in the set rather than in the skill. The drop in correct verdicts in the second run comes from there, and from one answer where the new rule (a `suffisant` verdict requires all requirements to be covered) corrects an inconsistency of the first run.
- *Variance.* 22 verdicts out of 24 are identical from one run to the other: the bench is stable enough to compare versions of the skill on verdicts. It is not yet stable on the recall of sources (0.67 → 0.50 on the holdout with no change to retrieval). This recall also underestimates the answers: most gaps come from an equivalent source (the `bench` row or the table passage rather than the expected text section).

No threshold is set yet: what is missing is a run with hybrid retrieval and reranking, larger sets (40 development and 60 held-out questions per base, written or reviewed by a third party) and the human ratings.

6.10 A second corpus: the drone base

The drone base (section 3.3) has its own question sets, written with the same protocol on its first 73 papers: 20 development questions, 20 held-out, 8 in French and two sets of 10 out-of-corpus questions. Thresholds are those of the fpga base, fixed before the first run. Table 15 compares the two bases (runs of 03/10/2026 at commit 3925690).

Table 15: Evaluation bench on the two bases that have question sets, on 03/10/2026 (recall@5 unless stated).

Measure	fpga (101 papers)	drone (87 papers)
Passages	8 601	5 361
Text coverage: mean / minimum	0.90 / 0.57	0.91 / 0.77
Tables with a body	0.78	0.79 (0.90 on the 73 GROBID papers)
Header: title / year / DOI	0.87 / 0.47 / 0.65	0.92 / 0.64 / 0.70
`benchmarks.csv` values found in the cited table	0.70 (1 044)	0.62 (739)
Dev, hybrid / hybrid + reranking	0.80 / 0.90	0.90 / 1.00
Held-out, hybrid / hybrid + reranking	0.68 / 0.72 (25 q.)	0.90 / 0.95 (20 q.)
Held-out, MRR hybrid + reranking	0.62	0.89
French, hybrid + reranking	0.38	0.50
Low relevance (`cos`): out-of-corpus / in-corpus flagged	0.80 / 0.20	1.00 / 0.10
`verify`: exact / word replaced / number altered / other paper	1.00 / 0.97 / 0.00 / 0.00	1.00 / 0.82 / 0.03 / 0.00

The reading calls for caution:

- **The drone held-out set meets its threshold**, but one cannot conclude that retrieval works better there. The drone corpus is more heterogeneous (detection, tracking, neuromorphic, drone racing): a distinctive fact has fewer close neighbours than in a corpus centred on accelerators. Two expectations were also widened after reading the returned passages. On 20 questions, 0.05 is one question. Between 28/09 and 03/10, with 14 more papers and 836 more passages, hybrid loses one question on each English set, and reranking loses none.
- **Failures are of the same kind** as on the fpga base: a table (idle power of several boards) missed by every mode, and half of the French questions.
- **Low relevance remains unusable**: the cosine flags every out-of-corpus validation question, but also 2 of the 20 held-out questions, above the 5 % of false alarms tolerated.
- **Tables.** Running Docling on this base (29/09) raised the share of tables with a body from 0.71 to 0.90 and the values found in their table by 3 points, with a constant corpus: this is the first measured gain of Docling on these values. The share falls back to 0.79 on 87 papers because the 14 papers added since were added without their PDF: extracted by PyMuPDF without GROBID, they have no table with a body (0 out of 50) and cannot go through Docling.
- `verify` accepts one quotation with a changed number out of 30, and the corresponding threshold is no longer met (section 6.6).

The stage base. It has no question sets yet: the bench only measures its extraction (56 papers, 3 640 passages, run `bases/stage/eval/results/2026-10-03T2344-3925690.json`). Mean coverage is 0.91 (minimum 0.61 for the scanned reprint of Mahalanobis, 1936), but only 71 % of the 254 tables have a body, none having gone through Docling, and only 42 % of the 112 values of `benchmarks.csv` are found in the cited table (57 % anywhere in the text): these rows are the first to reread. It is also the only base where unrepaired ligatures remain (0.37 per 10 000 words). Without question sets of its own, the bench until now silently fell back on the fpga base's questions, whose recall makes no sense here: this fallback must become a warning.

6.11 Synthetic benchmark (`pdb synth`)

The previous sets are small and written by the author of the system. `bin/pdb synth` prepares a larger evaluation, kept apart from the working base (`.cache/bench/`):

- a *frozen corpus* of 200 papers, with neighbouring topics as distractors; question anchor passages are screened for FPGA, embedded AI and hardware acceleration, excluding introductions and background;
- *local generation* of questions by an open model (Qwen3.8-27B quantised to Q6, served by llama.cpp on the same RTX 5090), as schema-constrained JSON with a fixed seed; the identity of the served model and every quotation are checked;
- *validation without human labels*: two independent sessions of an LLM (Codex for the 200 reference questions, then Claude Code in non-interactive mode, `claude -p`) check each question;
- *quotas*: 40 single-passage, 40 numerical, 50 cross-paper, 30 within-paper, 20 table and 20 missing-evidence questions; a 40 % development, 60 % test split over disjoint paper sets, balanced between English and French, and 40 questions declined into translated and terse variants;
- *ablations*: the four retrieval modes of the GPU engine and the two CPU modes of `pdb search` on the same questions, then agent variants run in a sandbox (bubblewrap, read-only tools, read-only corpus), scored by an LLM judge.

Automatically produced notes and numerical transcriptions form a separate *enriched* corpus, never used to write reference answers; a benchmark row is kept there only if each of its numbers appears in the cited section. The protocol and its tests are in place; its results are not available yet and are therefore not reported here.

6.12 Threats to validity

- The sets are small: on 25 questions, a difference of 0.04 is a single question, and no difference below two questions is readable. The holdout tuned no parameter, but it was written, like the dev set, by the author of the system. Questions written by a third party are needed to confirm the differences. The second corpus (section 6.10) shows that the trends are not tied to a single corpus, but its questions were written by the same author.
- The sets did not follow the growth of the bases: written for 45 and 73 papers, they now query less than half of the fpga corpus, and the stage base has none.
- Expected sections are chosen by text pattern. This is an approximation: a correct answer phrased differently, or found in a paper added after the set was written, counts as a failure.
- The ground truth for tables only covers values taken into `benchmarks.csv`, and the rate of tables "with a body" says nothing about the accuracy of the cells.

- The written answers are scored automatically on 24 questions, with lexical retrieval only, and the human ratings have not been assigned yet: the validity of the citations is checked, not whether they support the sentence.
- The comparison of the two engines is on the 28-paper corpus and on the original `pdb rag` chain; it was not redone with the engine built into `pdb search`, nor on the current corpus.
- The savings from delegation rest on an estimate, calibrated by a single paired trial, on a single task, with no scoring of quality.
- The synthetic benchmark will have its own biases: questions generated by an LLM, validated and scored by other LLMs. It will measure this frozen corpus and this task distribution, not research work in general.

7 Limitations and outlook

- **Scaling.** From 45 to 101 papers, the holdout recall@5 with reranking goes from 0.80 to 0.72, below the threshold. Before any tuning, the effect of the corpus must be separated from that of the code, by replaying the sets on the common sub-corpus and the old code on the current corpus. The next plan is specified: question sets of each base's own, with no silent fallback; a size proportional to the corpus (a holdout of at least 40 questions or half the number of papers); quotas per axis of the base and per question type (tables, several papers); questions proposed by the synthetic-benchmark chain and reviewed by a human; a new reference run each time the corpus grows by 25 %; and bootstrap confidence intervals, so that a difference is flagged only if it falls outside them.
- **Tables.** They remain the most fragile part of extraction: 22 % of the tables of the fpga base have no body, about 40 % of those that have one are flagged by a quality signal, and OCR'd tables are of uneven quality. Tables are hard to retrieve, now for a vocabulary reason (header abbreviations) more than for chunking; a per-base glossary of abbreviations or multiple queries are the leads. Still to do: a skill to reread flagged tables, starting with those cited by `benchmarks.csv`, a ground truth of about twenty hand-annotated tables, and OCR targeted at the table area rather than the whole PDF. A re-extraction (`ingest --reparse`) also undoes some ligature repairs: to be fixed before any mass re-extraction.
- **Comparisons.** `bench --comparable` says which rows compare along one axis. A `pdb compare` command, specified, will compare along two axes (throughput versus power, at equal metric and dataset), give a confidence level per group and explain each excluded row. The optional columns remain to be filled: no row carries its passage yet.
- **Atomic evidence.** Only one paper has its `claims.jsonl`. The others remain to be produced, to be indexed as pointers to their passage, and to be read by `/paper-ask`, which should gain on synthesis questions.
- **Answers.** Missing are a run of the answers with hybrid retrieval and reranking, larger sets reviewed by a third party, sets for the drone and stage bases, and the human ratings.
- **Out-of-corpus questions.** No retrieval score reliably says that the base does not cover the topic; the agent's sufficiency verdict manages it on the 6 questions tested, which remains to be confirmed on a larger set.
- **Verification.** `verify` rejects changed numbers when the number and its neighbours are not found elsewhere in the passage, but not in the opposite case (section 6.6), and a word replaced far from any number still passes as a paraphrase (`APPROX`). The check of `rag check-answer`, on the GPU route, does not check numbers yet.

- **Metadata.** A publisher template DOI (MDPI) is not yet rejected at extraction; it is fixed by hand in the base, but a new paper from the same publisher would inherit it.
- **Language.** The corpus and the default model are in English, and CPU retrieval loses a lot in French (recall@5 of 0.38 with reranking). The GPU engine retrieved all 8 French questions, at the price of a GPU. Specified leads: systematically translate the query and merge the results, evaluate a multilingual model alongside the current one, and raise the French set to 30 questions per base.
- **Provenance.** Nothing keeps track of the state of the corpus on which an answer was written: after a reindex, one cannot know what it was based on. A corpus fingerprint and an answer log are planned.
- **Latency.** About 7 s per cold `search`, versus 0.1 s warm, mostly due to the PyTorch import and model loading; `show` and `verify`, which do without them, answer in under a second. A resident process, for example an MCP server exposing the same commands, would pay this cost only once and would let other applications use the base. The same argument applies to the GPU engine, which reloads its models for each query (3 to 4 s).
- **Genericity.** The same code serves three bases (section 3.3), one of them with no hardware measurement at all, and a new one is created with `pdb base new`. The mandatory columns of `benchmarks.csv` (FPS, GOPS, FPGA resources) remain marked by the original domain: out of 100 rows, the stage base has no FPGA resource and two throughputs; it reports its results as accuracy or prediction error. The optional columns and the per-base `plot` block made the schema more flexible without making it configurable.

8 Conclusion

PaperDatabase shows that a modest research corpus can be made usable by an LLM agent without a database or an external service: well-structured text files, a table of sourced numbers and a deterministic hybrid search are enough. Generation stays with the agent, but everything it quotes can be replayed and checked by a command. The cost of a sourced answer drops by an order of magnitude compared with reading full texts, and delegating reading to less expensive subagents reduces the bill further, by 41 % in the single paired trial.

The evaluation bench showed the value of measuring what one believes settled: an optimistic question set, a warning that never fired, a check that let wrong numbers through, and then, on remeasuring, a retrieval that loses recall when the corpus doubles. This last point is the main open problem. The evaluation of the written answers brought the opposite result: where no score could tell that a question falls outside the corpus, the agent says so when it is made to confront each requirement of the question with the passages read. The same code serves three isolated bases, on which the bench finds the same strengths and the same weaknesses. Next steps are the diagnosis of the recall drop and question sets at the scale of the bases, questions written by a third party, human ratings of the answers, better retrieval within tables, comparison along two axes and a resident engine that avoids reloading models for each query.

References

Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, et al. Docling technical report. arXiv:2408.09869, 2024. Preprint.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM*

SIGIR Conference on Research and Development in Information Retrieval, pages 758–759, 2009. doi: 10.1145/1571941.1572114.

GROBID contributors. GROBID: machine learning software for extracting information from scholarly documents. `https://github.com/kermitt2/grobid`, 2008–2026. Logiciel, version 0.8.1 utilisée.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. MS MARCO: A human generated MAchine reading COmprehension dataset. In *Workshop on Cognitive Computation (CoCo@NIPS)*, 2016.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv:1901.04085, 2019. Preprint.

Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv:2205.01833, 2022. Preprint.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019. doi: 10.18653/v1/D19-1410.

Stephen E. Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. doi: 10.1145/3626772.3657878. Preprint arXiv:2309.07597 (2023).

A Glossary of acronyms

Acronym	Meaning
AI	artificial intelligence
API	Application Programming Interface
BF16	bfloat16, 16-bit floating-point format
BM25	Best Matching 25, scoring function of lexical retrieval
BRAM	Block RAM, on-chip memory of an FPGA
CPU	Central Processing Unit
CSV / TSV	Comma- / Tab-Separated Values
DOI	Digital Object Identifier, persistent identifier of a publication
DSP	Digital Signal Processing: arithmetic block (multiply-accumulate) of an FPGA
FPGA	Field-Programmable Gate Array
FPS	Frames Per Second
FTS5	Full-Text Search version 5, SQLite full-text index
GOPS	Giga Operations Per Second
GPU	Graphics Processing Unit
GROBID	GeneRation Of BIbliographic Data, structure extractor for PDF papers
INT8	8-bit integers (quantisation)
JSON	JavaScript Object Notation
LLM	Large Language Model
LUT	Look-Up Table, elementary logic cell of an FPGA
MCU	Microcontroller Unit
MCP	Model Context Protocol, protocol exposing tools to an LLM agent
MRR	Mean Reciprocal Rank of the first correct answer
MS MARCO	Microsoft MAchine Reading COmprehension, passage-retrieval dataset
nDCG	normalized Discounted Cumulative Gain, ranking quality
NMS	Non-Maximum Suppression, removal of duplicate detection boxes
OCR	Optical Character Recognition
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SHA-1	Secure Hash Algorithm 1, hash of a text
TEI	Text Encoding Initiative, XML output format of GROBID
YOLO	You Only Look Once, family of object detectors

B Glossary

Term	Meaning in this document
agent	LLM that calls tools (file reading, commands) to carry out a task; here Claude Code
benchmark row	one configuration reported by a paper (model, platform, precision, throughput, power…), with its source section, in `benchmarks.csv`
chunk, passage	section fragment indexed for retrieval, identified by `<paper>#<section>.<k>`
cross-encoder	model that scores a query and a passage together; used for reranking
requirement	what a complete answer must establish (value, platform, condition…); status `couvert`, `partiel` or `absent`
embedding	vector representing the meaning of a text; close texts have close vectors
reading note (`fiche`)	critical reading of a paper (`fiche.md`): problem, method, results, limitations, relevance
hook	command that Claude Code launches by itself at a given moment (here, at the end of each subagent)
holdout	reserved question set, never used to tune any parameter
dense / lexical index	index by embeddings (meaning) / by exact words (BM25, FTS5)
gap	area the corpus does not cover, recorded in `gaps.md`
`n/r`	"not reported": value absent from the paper, never estimated
power scope	what a power measurement covers (chip, board, estimate), `power_scope` column
atomic evidence (claim)	one statement of a paper, with its source passage and the exact excerpt that carries it, in `claims.jsonl`
preprint	paper released without peer review (arXiv)
recall@k	share of questions for which an expected section is among the top k passages
paper@k	share of questions for which an expected paper is among the top k passages
reranking	reordering of the top results by a more expensive model
skill	written procedure the agent follows for a task (`/paper-ask`, `/paper-fiche`…)
subagent	agent launched by the main agent for a delimited task, without its context, often on a less expensive model
toolflow	tool chain turning a neural network into a hardware accelerator (FINN, hls4ml…)
sufficiency verdict	at the head of an answer: `suffisant` (all requirements covered), `partiel` or `absent` (none)

C Commands

Command	Effect
`bin/pdb status [--tables]`	state of the base and next actions; table quality signals
`bin/pdb add <arxiv | doi | pdf>`	acquisition, ingestion and reindexing
`bin/pdb ingest [--force | --reparse]`	(re)extraction of the PDFs in `inbox/`
`bin/pdb ingest --reparse --tables docling [--ocr]`	bodiless tables handed to Docling
`bin/pdb reindex`	`INDEX.md`, `external_refs.csv`, passage index
`bin/pdb bench [filters]`	filters `benchmarks.csv`, source columns always shown
`bin/pdb bench --check | --comparable | --add-columns`	values found in their passage; comparable groups; optional columns
`bin/pdb claims <id> [--check]`	atomic evidence of a paper and its check
`bin/pdb paper <id | word>...`	identity of one or more papers
`bin/pdb search "<q>"`	ranked passages (hybrid, optional `--rerank`; `--gpu` / `--cpu`)
`bin/pdb gpu [on | off]`	state of the GPU engine; enabling it for the machine, back to the CPU
`bin/pdb show <chunk_id>`	exact text of a passage
`bin/pdb verify <chunk_id> "<quote>"`	checks a quotation
`bin/pdb cites | cited-by <id>`	internal citation graph
`bin/pdb discover`	candidates to add, via OpenAlex
`bin/pdb plot [bench | eval | corpus | all]`	static figures and `figures/manifest.json`
`bin/pdb plot --check | --stale`	`plot` block proposed for the base; stale figures only
`bin/pdb viz`	local corpus visualisation
`bin/pdb rag index | search | evidence | benchmarks`	GPU engine (Qwen3): index, search, evidence, numbers
`bin/pdb rag acquire | approve | check-answer`	open-access acquisition, import after approval, answer check
`bin/pdb synth <step>`	synthetic benchmark (corpus, generation, validation, ablations, report)
`bin/pdb base list | new | register | use`	registered bases, creation, registration, default base
`bin/pdb base info | check`	profile and axes of a base; effect of the `base.json` rules
`bin/pdb base scan | import <base>:<id>`	useful papers from another base; copy of a paper
`bin/pdb --base <name> <command>`	act on a base other than that of the current folder
`bin/pdb test`	unit tests
`bin/pdb eval [--extract | --rag | --speed]`	evaluation bench, by part or complete
`bin/pdb eval --save | --compare`	saves a report; compares with the latest one and the thresholds
`bin/pdb eval --rag --gpu`	retrieval evaluated with the GPU engine, same keys
`bin/pdb eval --questions | --answers <run>...`	questions only; scoring of the written answers
`bin/pdb agents [--report | --scan | --mark]`	subagent log and cost, savings from delegation
`/paper-add`, `/paper-fiche`	agent skills: addition; reading note and benchmark rows
`/paper-extract`	atomic evidence of a paper (`claims.jsonl`)
`/paper-ask`, `/paper-gaps`, `/paper-discover`	sourced answer with sufficiency verdict; gap map; discovery
`/eval-answers`	blindly produces the answers of a set, then has them scored
`/paper-research`	sourced answer with the GPU engine and open-access acquisition
`/base-profile`	proposed refinement of the base's profile

Installation. Python 3.12 and Docker (for GROBID):

```
bash bin/setup.sh                                # .venv and dependencies
.venv/bin/pip install -r requirements-embed.txt  # optional: dense retrieval
bin/pdb grobid start                             # GROBID on localhost:8070
bin/pdb add 2507.18174 && bin/pdb test
```